

Creating F Prime Projects

Kevin F. Ortega
NASA Jet Propulsion Laboratory

Copyright © 2026 California Institute of Technology.
Government sponsorship acknowledged.

Jet Propulsion Laboratory
California Institute of Technology

Project Structure

```
my-fprime-project/
├── lib
│   ├── fprime*
│   └── my-library*
├── MyFprimeProject
│   ├── Components
│   │   └── MyComponent
│   │       └── ...
│   ├── Deployments
│   │   └── MyDeployment
│   │       └── …
│   └── config
│       └── ...
├── requirements.txt
└── settings.ini
```

- Automatically create project structure with `fprime-bootstrap project`
- MyFprimeProject/Components should only have project specific components. Any component that could be shared across projects should be part of a library.
- Project specific configurations, including the overriding of F Prime's default configuration, to be done in the config directory.
- Libraries and nested libraries shall be listed in the settings.ini file

```
library_locations: ./lib/my-library:./lib/my-library2:./lib/my-library2/lib/NestedLibrary
```

- List additional python-pip dependencies. By default, it includes the lib/fprime/requirements.txt, which allows users to correctly install packages for their project.

Directories marked with * are git submodules.

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Reusable Libraries

```
my-library/
├── cmake
│   ├── platform
│   └── toolchain
├── MyLibrary
│   ├── Components
│   │   └── MyComponent
│   │       └── ...
│   ├── Subtopology
│   │   └── ...
│   ├── Types
│   │   └── ...
│   └── Ports
│       └── …
└── library.cmake
```

Two requirements:

- `library.cmake` is the only required file in an F Prime library

```
####
# library.cmake for MyLibrary
####
add_fprime_subdirectory("${CURRENT_CMAKE_LIST_DIR}/MyLibrary/Components/MyComponent")
add_fprime_subdirectory("${CURRENT_CMAKE_LIST_DIR}/MyLibrary/Subtopology")
add_fprime_subdirectory("${CURRENT_CMAKE_LIST_DIR}/MyLibrary/Types")
add_fprime_subdirectory("${CURRENT_CMAKE_LIST_DIR}/MyLibrary/Ports")
```

- Module directories shall be namespaced (i.e. `MyLibrary`) to ensure that modules do not collide with F´ and other libraries. Placing container directories (e.g. `Components`, `Svc`, `Drv`, etc.) directly at the root of the repository is *strongly* forbidden.

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Reusable Libraries: Toolchain and Platform

```
my-library/
├── cmake
│   ├── platform
│   └── toolchain
├── MyLibrary
│   ├── Components
│   │   └── MyComponent
│   │       └── ...
│   ├── Subtopology
│   │   └── ...
│   ├── Types
│   │   └── ...
│   └── Ports
│       └── …
└── library.cmake
```

- Developing for a new OS or Hardware platform requires a cmake directory at the library's root with a platform and toolchain directory.
- F´ libraries can provide one or more toolchains by placing <toolchain>.cmake files in a cmake/toolchain directory. ; each file is automatically discoverable and usable via `fprime-util generate <toolchain>`

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Reusable Libraries: Toolchain and Platform

```
my-library/
├── cmake
│   ├── platform
│   └── toolchain
├── MyLibrary
│   ├── Components
│   │   └── MyComponent
│   │       └── ...
│   ├── Subtopology
│   │   └── ...
│   ├── Types
│   │   └── ...
│   └── Ports
│       └── …
└── library.cmake
```

- Similarly for platforms, one or more <platform>.cmake files can exist in the cmake/platform directory. Users specify the platform their project should use with a `set(FPRIME_PLATFORM <platform>)` in their project's cmake file.

- Platform files are used to specify what OS implementation to use

```
register_fprime_config(
        PlatformVxWorks
    INTERFACE # No buildable files generated
    CHOOSES_IMPLEMENTATIONS
        Os_Console_VxWorks
        Os_Mutex_VxWorks
        Os_Queue_VxWorks
        Os_Task_VxWorks
        Fw_StringFormat_snprintf
        # No VxWorks Implementation
        Os_Cpu_Stub
        Os_File_Posix
        Os_Memory_Stub
        Os_RawTime_Stub
    BASE_CONFIG
)
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Reusable Libraries: Components

```
my-library/
├── cmake
│   ├── platform
│   └── toolchain
├── MyLibrary
│   ├── Components
│   │   └── MyComponent
│   │       └── ...
│   ├── Subtopology
│   │   └── ...
│   ├── Types
│   │   └── ...
│   └── Ports
│       └── …
└── library.cmake
```

- Different ways to organize:

```
├── MyLibrary
│   ├── Components
│   │   ├── Drv
│   │   │   └── MyDriver
│   │   └── Svc
│   │       └── MySvc
```

```
├── MyLibrary
│   ├── Drv
│   │   └── MyDriver
│   ├── Svc
│   │   └── MySvc
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Reusable Libraries: Subtopology, Types, Ports

```
my-library/
├── cmake
│   ├── platform
│   └── toolchain
├── MyLibrary
│   ├── Components
│   │   └── MyComponent
│   │       └── ...
│   ├── Subtopology
│   │   └── ...
│   ├── Types
│   │   └── ...
│   └── Ports
│       └── …
└── library.cmake
```

- Subtopology is a directory with a subtopology that connect multiple components together into a cohesive functional unit. They are used to encapsulate integration logic without duplicating wiring across multiple deployments.
  - Can be configurable per project
- The Ports directory has all the ports defined that are needed by the library's components.
- The Types directory has all the types defined that are needed by the library.
  - Array
  - Enums
  - structs

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Continuous Integration (CI) Platforms

```
my-library/ci/
├── pyproject.toml
├── setup.py
├── src/
│   └── <package_name>/
│       ├── __init__.py
│       └── plugins/
│           ├── __init__.py
│           └── <your_ci_plugin>.py
└── sample-configs/
    └── <board_name>.yml
```

- The ci directory for an F Prime library contains a Python plugin package that integrates with the fprime-ci system to enable hardware-in-the-loop testing and continuous integration for custom platforms.

- The pyproject.toml file is the standard, single configuration file for modern Python projects that consolidates project metadata, dependencies, and settings for various development tools

```
[project]
name = "my-library-ci"
dependencies = [ "fprime-ci" ]
[project.entry-points.fprime_gds]
your_ci = "<package_name>.plugins.<your_ci_plugin>:YourCi"
[tool.setuptools_scm]
root = ".."
```

- setup.py file is a simple setup script that uses the configuration in pyproject.toml. This file is here for legacy builds.

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Continuous Integration (CI) Plugin

```
my-library/ci/
├── pyproject.toml
├── setup.py
├── src/
│   └── <package_name>/
│       ├── __init__.py
│       └── plugins/
│           ├── __init__.py
│           └── <your_ci_plugin>.py
└── sample-configs/
    └── <board_name>.yml
```

- The src/<package_name>/plugins directory has your library's ci directory for an F Prime library contains a Python plugin package that integrates with the fprime-ci system to enable hardware-in-the-loop testing and continuous integration for custom platforms.

- The <your_ci_plugin>.py implements a class that extends the fprime_ci.ci.Ci abstract base class. Key features are:

- `class Key(fprime_ci.ci.Ci.Keys)`
- `def build(self, context: dict) -> dict`
- `def preload(self, context: dict)`
- `def load(self, context: dict)`
- `def launch(self, context: dict)`
- `def cleanup(self, context: dict)`
- `def get_name(cls) -> str`

- Reference: https://github.com/fprime-community/fprime-zephyr/blob/main/ci/src/fprime_zephyr/plugins/zephyr_ci.py

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Continuous Integration (CI) Board Configurations

```
my-library/ci/
├── pyproject.toml
├── setup.py
├── src/
│   └── <package_name>/
│       ├── __init__.py
│       └── plugins/
│           ├── __init__.py
│           └── <your_ci_plugin>.py
└── sample-configs/
    └── <board_name>.yml
```

- The sample-configs contains a yaml file that specifies a configurations for your board. File lists:
  - The integration and test scripts you want to run
  - Arguments to the GDS
  - Arguments to pytest
  - Any definitions needed by your plugin
- Sample configuration provides baseline that projects can copy over and update for their project.
- Verify your CI setup works by installing fprime-ci and manually running your CI
  - `fprime-ci --ci-selection <name-of-plugin> -c <config>.yml`

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Continuous Integration (CI) Non-Platform Libraries

- Use GitHub Actions
  - Execute on pull request to main or devel branch
    - Install dependencies e.g., fprime
    - Verify library builds
    - Verify all unit tests pass

- Use CAE Jenkins
  - Set up Jenkins by following instructions found here https://cae-jenkins2.jpl.nasa.gov/
  - Run your Jenkins on pull requests to main or devel branch
    - Install dependencies e.g., fprime
    - Verify library builds
    - Verify all unit tests pass

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Summary

- Use of `fprime-bootstrap project` to get the expected project structure

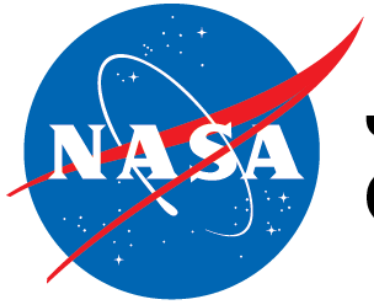
- Use reference projects to help set up your libraries and CI
  - https://github.com/fprime-community/fprime-zephyr-reference
  - https://github.com/fprime-community/fprime-sensors-reference
  - https://github.com/fprime-community/fprime-vxworks

- Look at the documentation https://fprime.jpl.nasa.gov and utilize its search function to find the answers to your questions

- Ask for help on https://github.com/nasa/fprime/discussions

- Internal sources of help:
  - Slack channel #fprime-users
  - F Prime office hours Wednesdays 1pm - 2pm

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.



Jet Propulsion Laboratory
California Institute of Technology